

# PostgreSQL Query Tuning Step-by-Step Guide

---

How to Tune a New Query from Scratch  
and Make It Run Faster in Production

Applicable Versions: PostgreSQL 15, 16, 17

Environment: Private Cloud VMs — Single Node & Streaming Replication

# Table of Contents

## 1. Introduction: The Query Tuning Methodology

- 1.1 The 8-Phase Query Tuning Workflow
- 1.2 Before You Begin: Essential Prerequisites

## 2. Phase 1: Capture the Baseline

- 2.1 Get the Execution Plan with EXPLAIN ANALYZE
- 2.2 Understanding EXPLAIN Output
- 2.3 Key Metrics to Record

## 3. Phase 2: Read the Execution Plan

- 3.1 How PostgreSQL Executes a Query
- 3.2 Node Types and What They Mean
- 3.3 Spotting Red Flags in Plans

## 4. Phase 3: Check Table Statistics

- 4.1 Why Statistics Matter
- 4.2 Refreshing Statistics
- 4.3 Adjusting Statistics Targets

## 5. Phase 4: Index Analysis & Creation

- 5.1 Choosing the Right Index Type
- 5.2 Composite Index Column Ordering
- 5.3 Covering Indexes and Index-Only Scans
- 5.4 Partial Indexes
- 5.5 Expression Indexes

## 6. Phase 5: Rewrite the SQL

- 6.1 Common Anti-Patterns and Fixes
- 6.2 JOIN Optimization
- 6.3 Subquery vs CTE vs Lateral Join
- 6.4 Pagination Optimization

## 7. Phase 6: Tune Planner Parameters

- 7.1 Session-Level Tuning Parameters
- 7.2 Influencing Join and Scan Strategies

7.3 Parallel Query Tuning

**8. Phase 7: Partitioning for Large Tables**

8.1 When to Partition

8.2 Range vs List vs Hash Partitioning

8.3 Partition Pruning Verification

**9. Phase 8: Validate & Monitor in Production**

9.1 Comparing Before vs After

9.2 Regression Detection

9.3 Ongoing Monitoring Setup

**10. Quick Reference: Tuning Decision Flowchart**

**11. Appendix: Common EXPLAIN Patterns & Solutions**

# 1. Introduction: The Query Tuning Methodology

When a new query arrives in a PostgreSQL production environment — whether from a new application feature, a report request, or an ad-hoc analytical need — it often performs poorly on the first execution. This is normal. The query has no execution history, the planner is making estimates based on table statistics, and the indexes in place may not align with the query's access patterns.

This guide provides a **systematic, repeatable methodology** for tuning any new query from scratch. Rather than relying on guesswork or random index creation, we follow an 8-phase workflow that moves from understanding the problem to implementing and validating the solution.

## 1.1 The 8-Phase Query Tuning Workflow

| Phase | Action                        | Goal                                         |
|-------|-------------------------------|----------------------------------------------|
| 1     | Capture the Baseline          | Record current execution plan and timing     |
| 2     | Read the Execution Plan       | Understand how PostgreSQL executes the query |
| 3     | Check Table Statistics        | Ensure the planner has accurate data         |
| 4     | Index Analysis & Creation     | Provide efficient data access paths          |
| 5     | Rewrite the SQL               | Eliminate anti-patterns and reduce work      |
| 6     | Tune Planner Parameters       | Guide the optimizer to better choices        |
| 7     | Partitioning for Large Tables | Reduce data scanned via partition pruning    |
| 8     | Validate & Monitor            | Confirm improvement and prevent regression   |

### Key Principle

Always change ONE thing at a time, and re-run EXPLAIN ANALYZE after each change. This lets you attribute the improvement (or regression) to a specific action. Tuning is a disciplined, iterative process — not a shotgun approach.

## 1.2 Before You Begin: Essential Prerequisites

- Ensure **pg\_stat\_statements** is loaded (in `shared_preload_libraries`) and the extension is created.
- Enable **track\_io\_timing = on** so that EXPLAIN (ANALYZE, BUFFERS) shows actual I/O time.

- Set **log\_min\_duration\_statement** to capture slow queries in the PostgreSQL log (e.g., 500ms to start).
- Have **auto\_explain** loaded if you want automatic plan capture for queries above a threshold.
- Confirm that autovacuum and ANALYZE are running properly — stale statistics is the number one cause of bad plans.

```
-- Verify prerequisites
SHOW shared_preload_libraries; -- Should include pg_stat_statements
SHOW track_io_timing;          -- Should be 'on'

-- Create the extension if not present
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

## PHASE 1

## 2. Phase 1: Capture the Baseline

Before you change anything, you must establish a **baseline measurement**. Without this, you have no way to prove that your tuning made things better (or worse). This is the most important discipline in query tuning.

### 2.1 Get the Execution Plan with EXPLAIN ANALYZE

The **EXPLAIN ANALYZE** command actually executes the query and reports the real execution plan along with actual timings. This is the single most important tool in your tuning toolkit.

```
-- The gold standard: EXPLAIN with ANALYZE, BUFFERS, and timing
EXPLAIN (ANALYZE, BUFFERS, COSTS, TIMING, FORMAT TEXT)
SELECT o.order_id, o.order_date, c.name, p.product_name,
       oi.quantity, oi.unit_price
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.order_date >= '2025-01-01'
      AND o.status = 'completed'
ORDER BY o.order_date DESC
LIMIT 50;
```

#### Safety Warning

EXPLAIN ANALYZE actually executes the query. For INSERT, UPDATE, DELETE queries, wrap them in a transaction and ROLLBACK to avoid modifying data: BEGIN; EXPLAIN ANALYZE UPDATE ...; ROLLBACK;

### 2.2 Understanding EXPLAIN Output

Each line in the EXPLAIN output represents a **plan node**. The key fields are:

| Field               | Meaning                              | What to Look For                                        |
|---------------------|--------------------------------------|---------------------------------------------------------|
| cost=startup..total | Estimated cost in arbitrary units    | High total cost relative to similar queries             |
| rows=N              | Estimated rows this node will return | Compare with actual rows (big difference = stale stats) |
| actual time=X..Y    | Real time in ms (startup..total)     | Where time is actually being spent                      |

| Field                  | Meaning                             | What to Look For                             |
|------------------------|-------------------------------------|----------------------------------------------|
| actual rows=N          | Real rows returned                  | Mismatch with estimated rows is a red flag   |
| loops=N                | How many times this node executed   | High loop count with Nested Loop = expensive |
| Buffers: shared hit=X  | Pages found in shared_buffers cache | High hit count = cached; read = disk I/O     |
| Buffers: shared read=X | Pages read from disk                | Reduce this via better indexes or caching    |
| I/O Timings            | Actual disk read/write time         | Pinpoints I/O-bound nodes                    |

## 2.3 Key Metrics to Record

For every tuning iteration, record these metrics in a spreadsheet or document:

- **Total Execution Time** — the bottom line of EXPLAIN ANALYZE.
- **Planning Time** — time spent by the optimizer choosing a plan.
- **Total Buffers Read** — sum of shared read across all nodes (measures physical I/O).
- **Total Buffers Hit** — sum of shared hit (measures cache effectiveness).
- **Rows Removed by Filter** — rows fetched but discarded (indicates missing index).
- **Top 3 Costliest Nodes** — which plan nodes consume the most time.

### Pro Tip: Use EXPLAIN Output Visualizers

Paste your EXPLAIN output into tools like [explain.dalibo.com](https://explain.dalibo.com), [explain.depesz.com](https://explain.depesz.com), or [pgMustard](https://pgmustard.com) for visual analysis. These tools highlight the expensive nodes and show row estimate accuracy, making it much easier to spot problems.

## PHASE 2

## 3. Phase 2: Read the Execution Plan

### 3.1 How PostgreSQL Executes a Query

PostgreSQL reads the execution plan from **innermost (deepest indentation) to outermost**. Each child node feeds data upward to its parent. Understanding this flow is critical for identifying which node is the bottleneck.

The planner chooses between different strategies for each operation: sequential scan vs index scan for accessing tables, nested loop vs hash join vs merge join for combining tables, and sort vs incremental sort for ordering. Your tuning efforts will influence these choices.

### 3.2 Node Types and What They Mean

| Node Type                 | What It Does                                                | Performance Implication                                                                          |
|---------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Seq Scan                  | Reads every row in the table                                | Acceptable for small tables or when most rows needed; BAD on large tables with selective filters |
| Index Scan                | Uses index to find matching rows, then fetches from table   | Good for selective queries returning < 10-15% of rows                                            |
| Index Only Scan           | Reads entirely from the index (no table access)             | Best possible scan — all needed columns in the index                                             |
| Bitmap Index Scan         | Builds a bitmap of matching pages, then fetches             | Good for medium selectivity; combines multiple indexes                                           |
| Nested Loop               | For each outer row, scans inner relation                    | Fast if inner has good index; disastrous without one                                             |
| Hash Join                 | Builds hash table from smaller relation, probes with larger | Good for equi-joins on larger datasets; needs work_mem                                           |
| Merge Join                | Merges two pre-sorted inputs                                | Efficient when both inputs are already sorted (via indexes)                                      |
| Sort                      | Sorts data in memory or on disk                             | Disk-based sort (external merge) is very slow; increase work_mem                                 |
| Aggregate / HashAggregate | Computes GROUP BY / aggregates                              | HashAggregate needs work_mem; GroupAggregate needs sorted input                                  |



| Node Type        | What It Does                                      | Performance Implication                                     |
|------------------|---------------------------------------------------|-------------------------------------------------------------|
| Materialize      | Caches results of a subplan for reuse             | Watch for high loop counts on materialized plans            |
| Memoize (PG 14+) | Caches parameterized inner results in Nested Loop | Reduces repeated computation; check cache hit ratio in plan |

### 3.3 Spotting Red Flags in Plans

These are the patterns that indicate a query needs tuning. Identify which of these apply to your query before moving to the next phase:

| Red Flag                                                   | Example in EXPLAIN                           | Likely Cause                                                |
|------------------------------------------------------------|----------------------------------------------|-------------------------------------------------------------|
| <b>Seq Scan on a large table with a selective filter</b>   | Rows Removed by Filter: 950000 (out of 1M)   | Missing index on the filtered column(s)                     |
| <b>Actual rows vastly different from estimated rows</b>    | rows=100 ... actual rows=500000              | Stale statistics; run ANALYZE or increase statistics target |
| <b>Nested Loop with high loop count and no inner index</b> | Nested Loop (actual loops=10000) -> Seq Scan | Missing index on the join column of the inner table         |
| <b>Sort node using external merge (disk)</b>               | Sort Method: external merge Disk: 256MB      | work_mem too low; or add an index matching ORDER BY         |
| <b>Hash Join with Batches &gt; 1</b>                       | Batches: 16 (originally 1)                   | work_mem too small to hold hash table; increase work_mem    |
| <b>Large Buffers: shared read</b>                          | Buffers: shared read=50000                   | Cold cache or table too large for buffers; improve indexing |
| <b>Filter removing 90%+ of fetched rows</b>                | Rows Removed by Filter: 9500 of 10000        | Index should push the filter into the scan, not post-filter |

## PHASE 3

## 4. Phase 3: Check Table Statistics

### 4.1 Why Statistics Matter

PostgreSQL's query planner uses **table statistics** (stored in `pg_statistic`) to estimate how many rows each operation will return. These estimates drive every decision the planner makes — which join strategy to use, whether to use an index or a sequential scan, and how to order operations. If the statistics are wrong, the entire plan can be catastrophically suboptimal.

A common scenario: a new table was just loaded with data, but `ANALYZE` has not yet run. The planner assumes the table has far fewer rows than it actually does, picks a Nested Loop join instead of a Hash Join, and the query takes 100x longer than it should.

### 4.2 Refreshing Statistics

#### Step 1: Check when statistics were last updated

```
SELECT schemaname, relname, n_live_tup,  
       n_mod_since_analyze,  
       last_analyze, last_autoanalyze  
FROM pg_stat_user_tables  
WHERE relname IN ('orders', 'customers', 'order_items', 'products')  
ORDER BY n_mod_since_analyze DESC;
```

If `n_mod_since_analyze` is high relative to `n_live_tup`, the statistics are stale.

#### Step 2: Run `ANALYZE` on all tables in the query

```
ANALYZE VERBOSE orders;  
ANALYZE VERBOSE customers;  
ANALYZE VERBOSE order_items;  
ANALYZE VERBOSE products;  
  
-- Then re-run your EXPLAIN ANALYZE and compare with baseline.  
-- If row estimates improve dramatically, this was the problem.
```

#### Step 3: Re-run `EXPLAIN ANALYZE` and compare

After `ANALYZE`, re-run the exact same `EXPLAIN ANALYZE`. If the **estimated rows** now closely match **actual rows**, and the plan changes (e.g., from Nested Loop to Hash Join), then stale statistics was the primary issue.

## 4.3 Adjusting Statistics Targets

For columns with non-uniform data distributions (e.g., a status column where 99% of rows are 'completed' and 1% are 'pending'), the default statistics target of 100 may not capture the distribution accurately enough. Increasing the target gives the planner more detail:

```
-- Increase statistics granularity for specific columns
ALTER TABLE orders ALTER COLUMN status SET STATISTICS 500;
ALTER TABLE orders ALTER COLUMN order_date SET STATISTICS 500;

-- Must re-ANALYZE after changing the target
ANALYZE orders;

-- Check the distribution the planner sees:
SELECT most_common_vals, most_common_freqs,
       histogram_bounds, n_distinct
FROM   pg_stats
WHERE  tablename = 'orders' AND attname = 'status';
```

### When to Increase Statistics Target

Increase STATISTICS for columns used in WHERE, JOIN, GROUP BY, or ORDER BY that have skewed distributions, a large number of distinct values, or when EXPLAIN still shows large row estimate errors after a regular ANALYZE. Values of 500–1000 are appropriate for critical columns. The maximum is 10,000 but rarely needed.

## PHASE 4

## 5. Phase 4: Index Analysis & Creation

After confirming that statistics are fresh and the planner still chooses a suboptimal plan (typically Seq Scan where Index Scan would be better), the next step is to analyze and create appropriate indexes.

### 5.1 Choosing the Right Index Type

| Index Type       | Best For                                                             | Example Use Case                                            |
|------------------|----------------------------------------------------------------------|-------------------------------------------------------------|
| B-tree (default) | Equality (=) and range (<, >, BETWEEN) comparisons, ORDER BY, UNIQUE | WHERE customer_id = 123; WHERE date BETWEEN ...             |
| Hash             | Equality-only lookups (=); smaller than B-tree for this case         | WHERE session_token = 'abc123'                              |
| GIN              | Full-text search, JSONB containment, array operations                | WHERE tags @> '{urgent}'; WHERE doc @@ to_tsquery(...)      |
| GiST             | Geometric/spatial data, range types, nearest-neighbor                | WHERE location <-> point(x,y) < 100                         |
| BRIN             | Very large, naturally ordered tables (timestamps, sequential IDs)    | WHERE created_at > '2025-01-01' on a 1B-row append-only log |

For the vast majority of OLTP query tuning, **B-tree indexes** are the correct choice. Use specialized types only when the data pattern or operator demands it.

### 5.2 Composite Index Column Ordering

The order of columns in a composite index is **critically important**. PostgreSQL can only use the leading (leftmost) columns of an index for lookups. The rule of thumb is:

1. **Equality columns first** — columns compared with = go on the left.
2. **Range/inequality columns next** — columns with >, <, BETWEEN go after equality columns.
3. **ORDER BY columns last** — if they match the sort direction.

```
-- Query pattern:
WHERE status = 'completed'      -- equality
  AND order_date >= '2025-01-01' -- range
ORDER BY order_date DESC

-- Optimal index (equality first, then range/sort):
CREATE INDEX CONCURRENTLY idx_orders_status_date
  ON orders (status, order_date DESC);

-- WRONG order (range first blocks equality optimization):
-- CREATE INDEX ON orders (order_date DESC, status); -- BAD
```

### The EqualRange-Sort Rule

Think of it as the ERS rule: Equality first, Range second, Sort third. This ordering maximizes the number of index columns PostgreSQL can use efficiently. Any column to the right of the first range column can only be used for filtering, not seeking.

## 5.3 Covering Indexes and Index-Only Scans

An **Index-Only Scan** is the fastest possible table access method because PostgreSQL reads data entirely from the index without touching the table heap. To achieve this, all columns referenced in the query (SELECT, WHERE, JOIN, ORDER BY) must be present in the index.

PostgreSQL supports **INCLUDE** columns that are stored in the index leaf pages but not in the B-tree structure — perfect for covering additional SELECT columns:

```
-- Query: SELECT order_id, total_amount
--         FROM orders WHERE status = 'completed'
--         AND order_date >= '2025-01-01'

-- Covering index with INCLUDE:
CREATE INDEX CONCURRENTLY idx_orders_covering
  ON orders (status, order_date DESC)
  INCLUDE (order_id, total_amount);

-- Verify Index-Only Scan in EXPLAIN:
-- -> Index Only Scan using idx_orders_covering
--   Heap Fetches: 0      <-- ideal: no table access needed
```

### Heap Fetches and Visibility Map

Even with a covering index, PostgreSQL may show Heap Fetches > 0 if the visibility map is not up to date. Running VACUUM on the table updates the visibility map, which allows the planner to skip heap fetches. If Heap Fetches are high, run VACUUM on the table.

## 5.4 Partial Indexes

A **partial index** indexes only a subset of rows that match a WHERE condition. This is extremely effective when your query targets a specific subset of data:

```
-- Only 5% of orders are 'pending', but queries always filter on them
CREATE INDEX CONCURRENTLY idx_orders_pending
  ON orders (order_date, customer_id)
  WHERE status = 'pending';

-- This index is much smaller than a full index and faster to scan.
-- The query's WHERE must match or imply the index's WHERE clause.
```

## 5.5 Expression Indexes

If your query applies a function to a column in the WHERE clause, a regular B-tree index on that column will **not** be used. You need an expression index:

```
-- This query CANNOT use a regular index on email:
SELECT * FROM users WHERE lower(email) = 'john@example.com';

-- Solution: expression index
CREATE INDEX CONCURRENTLY idx_users_email_lower
  ON users (lower(email));

-- Similarly for date extraction:
-- WHERE EXTRACT(year FROM created_at) = 2025
CREATE INDEX CONCURRENTLY idx_created_year
  ON orders (EXTRACT(year FROM created_at));

-- BETTER alternative: rewrite the query to use range:
-- WHERE created_at >= '2025-01-01' AND created_at < '2026-01-01'
-- This can use a regular B-tree index on created_at.
```

**PHASE 5**

## 6. Phase 5: Rewrite the SQL

Sometimes the most effective tuning is not adding indexes but rewriting the SQL itself. Certain query patterns inherently prevent the optimizer from choosing good plans. This phase focuses on identifying and fixing those anti-patterns.

### 6.1 Common Anti-Patterns and Fixes

#### Anti-Pattern 1: Functions on Indexed Columns in WHERE

```
-- BAD: Function on column prevents index use
SELECT * FROM orders
WHERE DATE(created_at) = '2025-03-01';

-- GOOD: Use a range comparison instead
SELECT * FROM orders
WHERE created_at >= '2025-03-01'
AND created_at < '2025-03-02';
```

#### Anti-Pattern 2: Implicit Type Casting

```
-- BAD: Column is varchar but compared with integer
-- PostgreSQL casts EVERY row, preventing index use
SELECT * FROM products WHERE product_code = 12345;

-- GOOD: Match the type
SELECT * FROM products WHERE product_code = '12345';
```

#### Anti-Pattern 3: SELECT \* When You Only Need a Few Columns

```
-- BAD: Fetches all columns, prevents Index-Only Scan
SELECT * FROM orders WHERE status = 'pending';

-- GOOD: Select only needed columns
SELECT order_id, order_date, total_amount
FROM orders WHERE status = 'pending';
```

#### Anti-Pattern 4: NOT IN with NULLable Columns

```
-- BAD: NOT IN fails with NULLs and scans everything
SELECT * FROM orders
WHERE customer_id NOT IN (SELECT customer_id FROM blocklist);

-- GOOD: Use NOT EXISTS instead
SELECT * FROM orders o
WHERE NOT EXISTS (
    SELECT 1 FROM blocklist b
    WHERE b.customer_id = o.customer_id
);
```

### Anti-Pattern 5: OR Conditions That Block Index Use

```
-- BAD: OR across different columns prevents single index scan
SELECT * FROM orders
WHERE customer_id = 100 OR product_id = 500;

-- GOOD: Use UNION ALL for independent conditions
SELECT * FROM orders WHERE customer_id = 100
UNION ALL
SELECT * FROM orders WHERE product_id = 500
    AND customer_id != 100; -- avoid duplicates
```

## 6.2 JOIN Optimization

- Ensure **join columns are indexed** on both sides of the join, especially the inner (right) table.
- Ensure **join column data types match exactly** — a join between integer and bigint forces a cast on every row.
- **Filter early:** push WHERE conditions as close to the base tables as possible, reducing the number of rows entering the join.
- For joins with many tables (5+), consider **set join\_collapse\_limit** and **from\_collapse\_limit** higher if the planner is not exploring enough join orders.

## 6.3 Subquery vs CTE vs Lateral Join

| Pattern           | Behavior in PostgreSQL                                                             | When to Use                                                                 |
|-------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Subquery in WHERE | Can be flattened into main query by planner; eligible for index use                | Prefer for simple existence checks (EXISTS) and IN lists                    |
| CTE (WITH clause) | PG 12+: planner can inline non-recursive CTEs. Use MATERIALIZED to force isolation | Use for readability; use MATERIALIZED only when you need optimization fence |



| Pattern      | Behavior in PostgreSQL                                        | When to Use                                                          |
|--------------|---------------------------------------------------------------|----------------------------------------------------------------------|
| LATERAL JOIN | Runs subquery once per outer row with access to outer columns | Top-N per group, correlated lookups; very powerful with LIMIT inside |

## 6.4 Pagination Optimization

The standard **OFFSET/LIMIT** pattern becomes extremely slow on large offsets because PostgreSQL must scan and discard all rows before the offset. Use **keyset pagination** (also called cursor-based pagination) instead:

```
-- BAD: OFFSET-based (slow at high offsets)
SELECT * FROM orders ORDER BY order_id LIMIT 20 OFFSET 100000;
-- Must scan 100,020 rows to return 20

-- GOOD: Keyset pagination (constant performance)
SELECT * FROM orders
WHERE order_id > :last_seen_order_id -- from previous page
ORDER BY order_id
LIMIT 20;
-- Uses index seek, scans only 20 rows regardless of page
```

## PHASE 6

## 7. Phase 6: Tune Planner Parameters

If statistics are fresh, indexes are in place, and the SQL is well-written, but the planner still chooses a suboptimal plan, you can influence its decisions through **session-level planner parameters**. These are tuning knobs that adjust cost estimates and plan strategy selection.

### 7.1 Session-Level Tuning Parameters

```
-- Adjust for the current session only (safe to experiment)

-- Increase memory for sorts and hash operations
SET work_mem = '256MB';

-- Tell the planner about SSD storage (lower random read cost)
SET random_page_cost = 1.1;

-- Adjust cache size estimate if shared_buffers + OS cache is large
SET effective_cache_size = '48GB';

-- Re-run EXPLAIN ANALYZE to see if the plan improves
```

| Parameter            | Default | Effect When Increased                                          | Typical Tuned Value      |
|----------------------|---------|----------------------------------------------------------------|--------------------------|
| work_mem             | 4MB     | Sorts and hash joins stay in memory; avoids temp files on disk | 64MB–512MB (per session) |
| random_page_cost     | 4.0     | Makes index scans relatively cheaper vs seq scans              | 1.1–1.5 (SSD)            |
| effective_cache_size | 4GB     | Planner assumes more data is cached; favors index scans        | 50–75% of VM RAM         |
| cpu_tuple_cost       | 0.01    | Makes processing rows more expensive; favors fewer rows        | Rarely changed           |
| cpu_index_tuple_cost | 0.005   | Makes index processing cheaper/costlier                        | Rarely changed           |

### 7.2 Influencing Join and Scan Strategies

In rare cases where the planner consistently picks the wrong join or scan strategy despite correct statistics, you can disable specific strategies **for the session** to test alternatives:

```
-- Temporarily disable strategies to test alternatives
SET enable_nestloop = off;      -- Force hash or merge join
SET enable_hashjoin = off;     -- Force nested loop or merge
SET enable_seqscan = off;      -- Force index scans (use cautiously!)
SET enable_sort = off;         -- Force index-based ordering

-- IMPORTANT: These are diagnostic tools, not production settings!
-- After testing, always reset:
RESET enable_nestloop;
RESET enable_seqscan;
-- Or: RESET ALL;
```

### Never Disable in postgresql.conf

These `enable_*` parameters should NEVER be set globally in `postgresql.conf`. They are diagnostic tools to understand what the planner would do with different strategies. If you find a better plan, investigate WHY the planner doesn't choose it (usually stale stats, wrong cost parameters, or missing indexes) and fix the underlying cause instead.

## 7.3 Parallel Query Tuning

PostgreSQL can parallelize queries by using multiple worker processes. This is especially beneficial for large sequential scans, aggregations, and hash joins. Key parameters:

```
-- Session-level parallel query tuning
SET max_parallel_workers_per_gather = 4; -- Workers per query node
SET parallel_tuple_cost = 0.01;         -- Lower = more parallelism
SET parallel_setup_cost = 100;          -- Lower = more parallelism
SET min_parallel_table_scan_size = '8MB'; -- Minimum table for parallel

-- Check if your query uses parallel workers:
-- Look for 'Workers Planned: N' and 'Workers Launched: N'
-- in the EXPLAIN output. If Workers Launched < Planned,
-- max_parallel_workers may be too low globally.
```

### PostgreSQL 17 Parallel Improvements

PostgreSQL 17 extends parallel query capabilities with better support for parallel hash joins and improved parallel append operations. If you're on PG 17, test with higher `max_parallel_workers_per_gather` values (4–8) on analytical queries.

## PHASE 7

## 8. Phase 7: Partitioning for Large Tables

### 8.1 When to Partition

Table partitioning divides a large table into smaller, physically separate sub-tables. The planner can then eliminate (prune) partitions that don't match the query's WHERE clause, dramatically reducing the amount of data scanned.

**Consider partitioning when:**

- The table exceeds **100 million rows** or **50 GB** in size.
- Queries almost always filter on a **specific column** (date ranges, region, tenant\_id).
- You need efficient **data lifecycle management** (dropping old partitions instead of DELETE).
- VACUUM and ANALYZE are too slow on the full table due to its size.

#### Do NOT Partition When

Partitioning adds complexity and is not free. Avoid it for tables under 10GB, tables where queries don't filter on the partition key, or when you have fewer than 20–50 planned partitions. Over-partitioning (1000+ partitions) can actually degrade planner performance.

### 8.2 Range vs List vs Hash Partitioning

| Strategy | Partition Key                               | Best For                                                | Example               |
|----------|---------------------------------------------|---------------------------------------------------------|-----------------------|
| Range    | Date/timestamp columns, sequential IDs      | Time-series data, logs, orders by date                  | BY RANGE (order_date) |
| List     | Discrete categories, status values, regions | Multi-tenant tables, region-based data                  | BY LIST (region_code) |
| Hash     | Evenly distribute by any column             | When no natural range/list key exists; parallel workers | BY HASH (customer_id) |

**Example: Range Partitioning by Month**

```
-- Create partitioned table
CREATE TABLE orders (
  order_id      bigserial,
  customer_id   int NOT NULL,
  order_date    date NOT NULL,
  status        varchar(20),
  total_amount  numeric(12,2)
) PARTITION BY RANGE (order_date);

-- Create monthly partitions
CREATE TABLE orders_2025_01 PARTITION OF orders
  FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
CREATE TABLE orders_2025_02 PARTITION OF orders
  FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
-- ... create future partitions proactively

-- Create indexes on EACH partition (or use indexing on parent):
CREATE INDEX ON orders (order_date, status);
-- This automatically creates matching indexes on all partitions
```

### 8.3 Partition Pruning Verification

After partitioning, verify that the planner is actually pruning partitions in your queries:

```
-- Ensure pruning is enabled:
SET enable_partition_pruning = on; -- default: on

-- Run EXPLAIN and look for pruned partitions:
EXPLAIN (ANALYZE)
SELECT * FROM orders
WHERE order_date >= '2025-03-01' AND order_date < '2025-04-01';

-- Should show: Partitions selected: 1 (out of N)
-- or only the relevant partition in the plan tree.
-- If ALL partitions are scanned, the WHERE clause does not
-- align with the partition key or runtime pruning is needed.
```

#### PostgreSQL 15-17 Partition Improvements

PostgreSQL 15 improved partition pruning for boolean partition keys and added better support for merge joins on partitioned tables. PostgreSQL 16 improved join planning with partitioned tables. PostgreSQL 17 further improved partition pruning for OR conditions and IN lists, and added better execution-time pruning.

## PHASE 8

## 9. Phase 8: Validate & Monitor in Production

### 9.1 Comparing Before vs After

After applying your tuning changes, do a structured comparison:

| Metric                  | Before Tuning     | After Tuning          | Target                            |
|-------------------------|-------------------|-----------------------|-----------------------------------|
| Execution Time          | (record baseline) | (record new)          | > 50% reduction or under SLA      |
| Planning Time           | (record baseline) | (record new)          | < 10ms for OLTP                   |
| Buffers Read (disk I/O) | (record baseline) | (record new)          | Significant reduction             |
| Buffers Hit (cache)     | (record baseline) | (record new)          | Should increase                   |
| Rows Removed by Filter  | (record baseline) | (record new)          | Should decrease or vanish         |
| Scan Type               | Seq Scan          | Index/Index Only Scan | Index-based for selective queries |

**Run the query at least 3 times** after tuning to account for caching. The first execution populates `shared_buffers`; subsequent executions reflect cached performance. Record both the cold-cache and warm-cache timings.

### 9.2 Regression Detection

After deploying the change to production, monitor for regressions:

```
-- Track the specific query in pg_stat_statements
SELECT queryid, calls, mean_exec_time, stddev_exec_time,
       min_exec_time, max_exec_time, rows
FROM pg_stat_statements
WHERE query ILIKE '%your_table_or_pattern%'
ORDER BY mean_exec_time DESC;

-- Reset statistics periodically to get fresh windows:
SELECT pg_stat_statements_reset();
```

- Compare **mean\_exec\_time** before and after your change.
- Watch **stddev\_exec\_time** — high variance means inconsistent performance (possible lock contention or plan flipping).

- Monitor **calls** count — ensure the application is actually using the optimized path.

### 9.3 Ongoing Monitoring Setup

- Use **auto\_explain** with `log_min_duration` to automatically capture plans for queries exceeding your SLA:

```
-- In postgresql.conf:  
shared_preload_libraries = 'auto_explain, pg_stat_statements'  
auto_explain.log_min_duration = '500ms'  
auto_explain.log_analyze = on  
auto_explain.log_buffers = on  
auto_explain.log_format = 'json' -- easy to parse
```

- Set up **pg\_stat\_statements** dashboards in Grafana showing top queries by `total_exec_time`, `mean_exec_time`, and `calls`.
- Create alerts for queries where **mean\_exec\_time** exceeds your SLA threshold (e.g., > 1 second).
- Review **pg\_stat\_user\_tables** weekly for tables with growing `seq_scan` counts on large tables.

## 10. Quick Reference: Tuning Decision Flowchart

Use this flowchart when you have a slow query and need to quickly determine the most likely fix:

| Step                          | Question                                                         | Action                                                                           |
|-------------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>1. Run EXPLAIN ANALYZE</b> | Does estimated rows differ from actual rows by > 10x?            | → <b>PHASE 3:</b> Run ANALYZE, increase STATISTICS target                        |
| <b>2. Check scan types</b>    | Is there a Seq Scan on a large table with a selective filter?    | → <b>PHASE 4:</b> Create appropriate index (B-tree, composite, partial)          |
| <b>3. Check filters</b>       | Does a node show 'Rows Removed by Filter' > 90% of fetched rows? | → <b>PHASE 4:</b> Index on the filtered column; or <b>PHASE 5:</b> rewrite query |
| <b>4. Check sort/hash</b>     | Is a Sort using 'external merge Disk' or Hash with Batches > 1?  | → <b>PHASE 6:</b> Increase work_mem (session or per-query)                       |
| <b>5. Check join strategy</b> | Is Nested Loop running with high loops and no inner index?       | → <b>PHASE 4:</b> Create index on inner table's join column                      |
| <b>6. Check anti-patterns</b> | Functions on columns? Implicit casts? SELECT *? OFFSET?          | → <b>PHASE 5:</b> Rewrite the SQL to eliminate anti-patterns                     |
| <b>7. Check table size</b>    | Is the table > 50GB / 100M rows with date-range filters?         | → <b>PHASE 7:</b> Implement table partitioning                                   |
| <b>8. Check cost settings</b> | Does the planner prefer Seq Scan despite good indexes on SSD?    | → <b>PHASE 6:</b> Lower random_page_cost to 1.1; increase effective_cache_size   |



## 11. Appendix: Common EXPLAIN Patterns & Solutions

### A. Pattern: Seq Scan with Filter

```
-> Seq Scan on orders (cost=0.00..25234.00 rows=500 width=80)
    (actual time=0.015..320.456 rows=487 loops=1)
    Filter: (status = 'pending')
    Rows Removed by Filter: 999513
    Buffers: shared hit=2000 read=12234
```

**Diagnosis:** Scanning 1M rows to find 487. The column *status* is not indexed.

**Fix:** CREATE INDEX CONCURRENTLY idx\_orders\_status ON orders (status); or a partial index WHERE status = 'pending'.

### B. Pattern: Nested Loop with Inner Seq Scan

```
-> Nested Loop (actual time=0.5..4500.3 rows=50000 loops=1)
    -> Index Scan on orders (actual rows=50000 loops=1)
    -> Seq Scan on order_items (actual rows=3 loops=50000)
        Filter: (order_id = orders.order_id)
```

**Diagnosis:** For each of 50,000 orders, PostgreSQL does a full table scan on order\_items. Total: 50,000 sequential scans.

**Fix:** CREATE INDEX CONCURRENTLY idx\_order\_items\_oid ON order\_items (order\_id);

### C. Pattern: Sort with Disk Spill

```
-> Sort (actual time=850.2..920.5 rows=500000 loops=1)
    Sort Key: order_date DESC
    Sort Method: external merge Disk: 128000kB
```

**Diagnosis:** work\_mem (default 4MB) is too small to sort 500K rows in memory, causing a slow disk-based sort.

**Fix:** SET work\_mem = '256MB'; for the session, or create an index on (order\_date DESC) to avoid sorting entirely.

### D. Pattern: Hash Join with Multiple Batches

```
-> Hash Join (actual time=200.5..1500.3 rows=100000 loops=1)
      Hash Cond: (o.customer_id = c.customer_id)
      -> Seq Scan on orders o (actual rows=1000000)
      -> Hash (actual rows=50000)
           Batches: 8 Memory Usage: 4096kB
```

**Diagnosis:** The hash table doesn't fit in work\_mem, forcing 8 batches (multiple passes over the data).

**Fix:** SET work\_mem = '128MB'; to allow the hash table to fit in a single batch. Target Batches: 1.

## E. Pattern: Index Scan with High Heap Fetches

```
-> Index Only Scan using idx_orders_covering on orders
      (actual time=0.05..15.3 rows=5000 loops=1)
      Heap Fetches: 4800
```

**Diagnosis:** Despite using Index Only Scan, 4800 of 5000 rows still need heap access because the visibility map is not up to date.

**Fix:** VACUUM orders; to update the visibility map. After VACUUM, Heap Fetches should drop to near 0.

## F. Pattern: Memoize with Low Cache Hit Ratio (PG 14+)

```
-> Memoize (actual time=0.01..450.2 rows=3 loops=50000)
      Cache Key: orders.customer_id
      Hits: 500 Misses: 49500 Evictions: 48000
```

**Diagnosis:** The Memoize cache has very low hits (1%) because the key has too many distinct values relative to cache size.

**Fix:** Increase work\_mem or switch to a Hash Join strategy. Test with SET enable\_memoize = off; to see if an alternative plan is faster.